

Introdução ao *JupyterLab*

Os exercícios do módulo são **notebooks** (`.ipynb`): documentos que misturam texto, código e o resultado de cada execução. O **JupyterLab** é a interface em que você os abre e roda, célula por célula. Nesta aula: abrir o JupyterLab, conhecer a tela, células de código e de texto, os dois modos e os atalhos, a ordem de execução e o kernel.

jupyter lab

Células

Markdown

Modo de edição × comando

Atalhos

Kernel

Formação Claude Code 2026 · Baseado em jupyterlab.readthedocs.io (Starting JupyterLab, The JupyterLab Interface, Notebooks), versão 4.6.4, consultado em outubro de 2026 · Tela ilustrativa com o notebook `01-roles.ipynb`

Um notebook é uma *lista de células*

CÉLULA DE CÓDIGO

Python que você roda; o resultado aparece logo abaixo dela.

CÉLULA MARKDOWN

Texto formatado: títulos, listas, negrito. Explica o que o código faz.

SAÍDA

O que a célula imprimiu. Fica **gravada no arquivo** ao salvar.

Ideal para engenharia de prompt: muda o prompt de uma célula, roda só ela e compara com a resposta de antes, sem rodar o resto.

Por isso os notebooks do módulo chegam com respostas já gravadas: é o que o modelo devolveu quando foram executados.

Do terminal *ao navegador*

Na pasta `Eng_prompt`, com o `venv` ativado (aula 02)

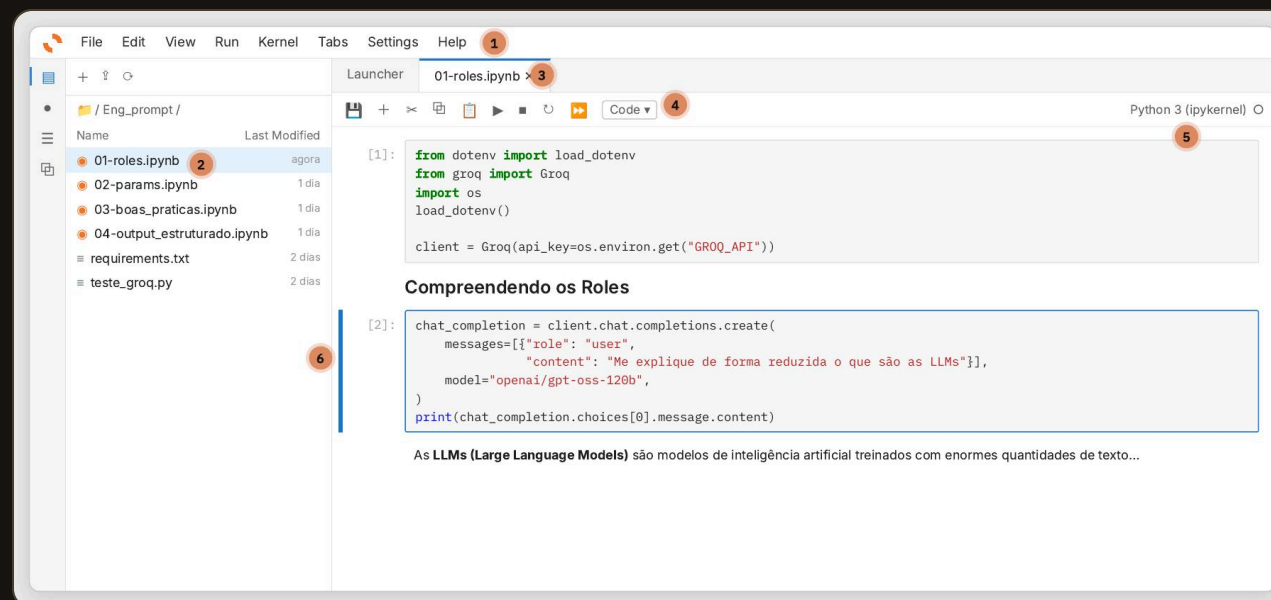
```
jupyter lab
```

- ✓ O JupyterLab abre sozinho no navegador
- ✓ O endereço fica no terminal, algo como `http://localhost:8888/lab?token=...`; se o navegador não abrir, copie e cole
- ✓ O **terminal precisa ficar aberto**: é ele que está rodando o servidor

Abra sempre **a partir da pasta do projeto**. A documentação recomenda não iniciar o JupyterLab na raiz do disco (`C:\` ou `/`), para não mexer em arquivos do sistema por engano.

Para encerrar: salve os notebooks, volte ao terminal e pressione `Ctrl` + `C` (confirme se ele perguntar). Também dá pelo menu **File > Shut Down**.

As partes *da interface*



JupyterLab com o notebook 01-roles.ipynb · reprodução ilustrativa

1 Menus: File, Run, Kernel...

2 Navegador de arquivos da pasta

3 Abas: cada notebook numa aba

4 Barra: salvar, ▶ rodar, ■ parar, ↺ reiniciar, tipo da célula

5 Kernel em uso: Python 3 (ipykernel)

6 Célula ativa (barra azul à esquerda)

Código *e* Markdown

Célula Markdown dos notebooks (como você digita)

Compreendendo os Roles

- User: Mensagem do usuário
- Assistant: Resposta do modelo
- **temperature**: controla a aleatoriedade

Trechos das células de texto dos notebooks 01-roles e 02-params.

- ✓ `###` vira título; `-` vira item de lista; **`texto`** vira negrito
- ✓ Ao rodar uma célula Markdown, ela aparece formatada
- ✓ Troque o tipo pela caixa **Code** ▼ da barra (marcador 4) ou pelos atalhos `M` (Markdown) e `Y` (código), no modo de comando
- ✓ Duplo clique numa célula Markdown volta a mostrar o texto cru

Dois modos, *atalhos diferentes*

ATALHO	FUNCIONA EM	FAZ
Shift + Enter	os dois	Roda a célula e vai para a próxima
Ctrl + Enter	os dois	Roda e fica na mesma célula
Alt + Enter	os dois	Roda e cria uma célula abaixo
Esc / Enter	—	Vai para o modo de comando / de edição
A / B	comando	Nova célula acima / abaixo
D D	comando	Apaga a célula (D duas vezes)
Z	comando	Desfaz a última operação com células
Ctrl + S	os dois	Salva o notebook

MODO DE EDIÇÃO

O cursor está dentro da célula: as teclas viram texto. Entra com

Enter

 ou clicando no código.

MODO DE COMANDO

A célula está selecionada, sem cursor: as teclas viram comandos (A, B, D D, M, Y). Entra com

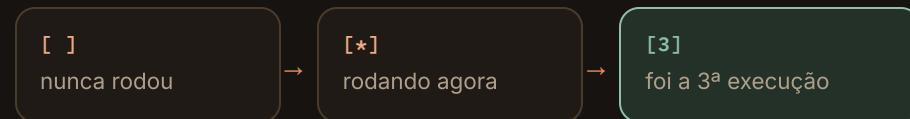
Esc

.

Digitou "a" e apareceu uma célula nova? Você estava no modo de comando.

No macOS, troque Ctrl por Cmd nos atalhos de salvar.

A ordem em que você roda *é a que vale*



- ✓ Variáveis criadas numa célula ficam na memória para as próximas
- ✓ O número entre colchetes mostra a **ordem real** das execuções, não a posição na tela
- ✓ Enquanto aparece `[*]`, espere: a chamada à API ainda não voltou

RODOU A CÉLULA DO PROMPT ANTES DA PRIMEIRA · SAÍDA REAL

```
NameError: name 'client' is not defined
```

Ao abrir um notebook, rode **primeiro a célula 1** (a do `load_dotenv()` e do `client`). Depois, as outras em qualquer ordem.

O *kernel*: o Python por trás do notebook

- ✓ O kernel é o processo Python que executa as células e guarda as variáveis
- ✓ Instalado dentro do venv, o JupyterLab usa o Python do venv: por isso o `import groq` funciona
- ✓ Cada notebook aberto tem o seu kernel

MENU KERNEL	QUANDO USAR
Interrupt Kernel (■)	Uma célula travou ou está demorando demais
Restart Kernel (↻)	Começar do zero: apaga todas as variáveis
Restart Kernel and Run All Cells	Conferir se o notebook roda inteiro, de cima para baixo

Mudou o `.env` ? Reinicie o kernel e rode a célula 1 de novo: a chave antiga continua na memória até lá.

Run All chama a API uma vez por célula. Num notebook com 7 chamadas, são 7 requisições e milhares de tokens da sua cota (aula 01).

O que fica *salvo*

- ✓ O `.ipynb` guarda o código, o texto e as saídas da última execução
- ✓ **Edit > Clear Outputs of All Cells** (ou **Kernel > Restart Kernel and Clear Outputs of All Cells**) limpa as respostas antes de compartilhar
- ✓ O JupyterLab cria a pasta `.ipynb_checkpoints/` com cópias de segurança; ela já está no `.gitignore` da aula 02

As saídas podem conter dados sensíveis. Se você imprimir a chave por engano (`print(os.environ["GROQ_API"])`), ela fica gravada no arquivo: limpe a saída antes de salvar e compartilhar.

Neste curso, ao lado de cada notebook há um `.md` e um `.pdf` com o mesmo conteúdo, para ler sem abrir o JupyterLab.

Seu primeiro *notebook*

1

Crie o notebook

No Launcher, clique em **Python 3 (ipykernel)** em Notebook. Renomeie para `teste.ipynb` (clique com o botão direito na aba).

2

Uma célula Markdown

Escreva `### Meu primeiro teste`, aperte `Esc` `M` e rode com `Shift` + `Enter`.

3

A célula de preparação

Cole a primeira célula dos notebooks (`load_dotenv()` + `Groq(...)`) e rode.

4

Um prompt

Crie a célula da chamada com uma pergunta sua e rode. Depois, reinicie o kernel e tente rodar só ela: veja o `NameError`.

Teste o que *ficou*

1. Qual atalho roda a célula e vai para a próxima?

A) Shift + Enter

B) Esc

C) D D

D) Ctrl + S

A.

2. [*] ao lado de uma célula indica que:

A) Deu erro

B) Ela está rodando

C) Nunca rodou

D) É Markdown

B.

3. "NameError: name 'client' is not defined" significa:

A) Chave inválida

B) A célula 1 não foi rodada nesta sessão

C) Sem internet

D) Limite estourado

B.

4. Reiniciar o kernel:

A) Apaga o arquivo

B) Apaga as variáveis da memória

C) Troca de modelo

D) Salva o notebook

B.

Rode, compare, *ajuste*

O JupyterLab abre com `jupyter lab` na pasta do projeto. Notebooks são células de código e de Markdown; o modo de comando tem atalhos próprios; a ordem de execução é a que vale; e o kernel guarda as variáveis até ser reiniciado. Com isso, você está pronto para os notebooks do módulo.

`jupyter lab`

`Shift + Enter`

`Esc · Enter`

`[1] [*]`

`Restart Kernel`

Próxima aula → 04 · Compreendendo as Roles

Fontes: jupyterlab.readthedocs.io/en/stable/getting_started/starting.html; jupyterlab.readthedocs.io/en/stable/user/interface.html; jupyterlab.readthedocs.io/en/stable/user/notebook.html (JupyterLab 4.6.4), consultados em outubro de 2026. Tela: reprodução ilustrativa (fonte em [scripts/aulas/telas/m39_a03_jupyterlab.html](#)). `NameError` gerado com Python 3.12.